

UltraTech

Web Application & Network Security Assessment

CONFIDENTIAL

Engagement Type	Black-box assessment
Primary Target	10.48.188.40
Web Service	http — ports 8081 (API), 31331 (web)
Host OS	Linux / Ubuntu
Tester	Saqr ElFirgany
Report Date	25 June 2026
Environment	TryHackMe — authorised lab

This document contains confidential information about the security posture of the assessed target. The engagement was performed in an isolated, authorised training environment (TryHackMe). The techniques documented here must only be used against systems you are explicitly authorised to test.

Table of Contents

1. Executive Summary	3
2. Scope & Methodology	3
3. Findings Summary	4
4. Attack Narrative (Kill Chain)	4
5. Detailed Findings	5
6. Privilege Escalation	7
7. Remediation Summary	7
8. Appendix A — Services & Testing Performed	8

1. Executive Summary

A black-box penetration test was performed against the UltraTech host to identify and exploit security weaknesses in its exposed services. The assessment resulted in a full compromise of the host: starting from an unauthenticated position, the tester achieved remote code execution through the internal REST API, recovered stored user credentials, and ultimately obtained **root** access.

The root cause of the compromise was an **OS command injection** vulnerability in the `/ping` endpoint of the Node.js API exposed on port 8081. This single flaw allowed arbitrary command execution as the web service account, which in turn exposed a world-readable credentials database. The stored passwords were protected with weak, unsalted MD5 hashes that were trivially cracked offline, and the recovered credentials were reused for SSH access — turning a web-layer flaw into a full interactive foothold and, subsequently, privilege escalation to root.

Overall Risk: Critical. A remote, unauthenticated attacker can fully compromise the host. The most urgent action is to remediate the command injection in the API (F-01); the credential-handling weaknesses (F-02, F-03, F-04) compound the impact and should be addressed in parallel.

2. Scope & Methodology

Scope

A single host was in scope, assessed from an unauthenticated, external (black-box) perspective with no prior credentials or knowledge of the application's internals.

Methodology

The assessment followed a structured, surface-first methodology, deliberately separating enumeration (mapping the attack surface) from exploitation:

- **Reconnaissance** — full port and service-version scanning to enumerate exposed services.
- **Enumeration** — content discovery and endpoint mapping on each web service; behavioural review of the API.
- **Attack-surface mapping** — each discovered entry point recorded as a hypothesis before any exploitation.
- **Exploitation** — confirmation of the highest-probability vulnerabilities to establish a foothold.
- **Post-exploitation** — credential recovery, lateral movement (SSH), and privilege escalation.

3. Findings Summary

ID	FINDING	SEVERITY	AFFECTED COMPONENT
F-01	OS command injection in API <code>/ping</code> endpoint	CRITICAL	Node.js API : 8081
F-02	Sensitive data exposure — world-readable credentials database	HIGH	API host filesystem
F-03	Weak, unsalted MD5 password hashes	HIGH	utech.db.sqlite
F-04	Credential reuse enabling SSH access	HIGH	SSH : 22
F-05	Verbose error / stack-trace information disclosure	MEDIUM	Node.js API : 8081
F-06	Information disclosure via <code>robots.txt</code> / sitemap	LOW	Web app : 31331

4. Attack Narrative (Kill Chain)

The following sequence describes the end-to-end path from initial reconnaissance to full host compromise:

1. Port scanning revealed two web services in addition to FTP and SSH: a Node.js/Express API on **8081** and a web application on **31331**.
2. Directory enumeration of the API surfaced an `/auth` and a `/ping` endpoint.
3. Requesting `/ping` without the expected parameter triggered a **verbose stack trace (F-05)** that disclosed the application's source path, the responsible line of code, and that the user-supplied `ip` parameter was passed into a `.replace()` call before being used.
4. This indicated the parameter was concatenated into a system `ping` command, which was confirmed as **OS command injection (F-01)**.
5. Command execution was used to list the application directory and read the **world-readable SQLite database (F-02)**, `utech.db.sqlite`, which contained user records.

6. The stored passwords were **unsalted MD5 hashes (F-03)** and were cracked offline, yielding valid credentials.
7. The recovered credentials were **reused for SSH (F-04)**, granting an interactive shell on the host.
8. Local privilege escalation was then performed to obtain **root**.

5. Detailed Findings

OS Command Injection in API `/ping` Endpoint

F-01 · CRITICAL

Severity: **CRITICAL** | Component: Node.js/Express API, port 8081 | Type: OS Command Injection (CWE-78)

Description

The `/ping` endpoint accepts a user-controlled `ip` query parameter and passes it into an operating-system `ping` command without adequate validation or sanitisation. Because the value is incorporated into a shell command, an attacker can append additional commands and have them executed by the server with the privileges of the API process.

Evidence

A benign request confirmed the endpoint executes `ping` against the supplied host:

```
GET /ping?ip=127.0.0.1
PING 127.0.0.1 (127.0.0.1) ... 1 packets received, 0% packet loss
```

Injecting a command separator allowed arbitrary commands (e.g. directory listing) to run on the host:

```
index.js  node_modules  package.json  package-lock.json  start.sh  utech.db.sqlite
```

Impact

Full arbitrary command execution on the host as the web-service account. This is the primary entry point that enabled the entire compromise chain, including credential theft and, ultimately, root access.

Remediation

- Do not pass user input to a shell. Replace shell invocation with a safe library call, or use an API that does not spawn a shell (e.g. argument-array execution rather than string concatenation).
- Strictly validate the `ip` parameter against an allow-list pattern for IPv4/IPv6 addresses; reject anything that does not match.
- Run the service under a least-privilege account with no unnecessary filesystem access.

Sensitive Data Exposure — World-Readable Credentials Database

F-02 · HIGH

Severity: HIGH | **Component:** API host filesystem | **Type:** Sensitive Data Exposure (CWE-538)

Description

The application stored its user database, `utech.db.sqlite`, inside the application directory where it was readable by the web-service account. Once command execution was achieved, the database was read directly and its contents dumped.

Evidence

```
CREATE TABLE users ( login Varchar, password Varchar, type Int )
r00t    f357a0c52799563c7c7b76c1e7543a32
admin   0d0ea5111e3c1def594c1684e3b9be84
```

Impact

Exposure of all application user accounts and their password hashes, enabling offline cracking and subsequent account takeover.

Remediation

- Store databases outside the web/application root and restrict file permissions to only the owning service account.
- Segregate the database from the application process where feasible, and apply least-privilege filesystem access.

Weak, Unsalted MD5 Password Hashes

F-03 · HIGH

Severity: HIGH | **Component:** utech.db.sqlite | **Type:** Use of a Broken/Risky Cryptographic Algorithm (CWE-327 / CWE-916)

Description

User passwords were stored as unsalted MD5 hashes. MD5 is fast and unsuitable for password storage; without a per-user salt, the hashes are vulnerable to rapid dictionary and rainbow-table attacks. Both recovered hashes were cracked offline.

Evidence

```
admin : mrsheafy
root  : n100906
```

Impact

Any attacker who obtains the database can recover plaintext passwords almost instantly, defeating the protection the hashing was intended to provide.

Remediation

- Replace MD5 with a modern, slow, salted password-hashing function such as bcrypt, scrypt, or Argon2.
- Force a password reset for all accounts once migrated, and enforce a strong password policy.

Credential Reuse Enabling SSH Access

F-04 · HIGH

Severity: HIGH | **Component:** SSH, port 22 | **Type:** Credential Reuse (CWE-522)

Description

Credentials recovered from the application database were also valid for SSH, allowing the tester to pivot from a constrained command-execution context to a full interactive shell on the host.

Evidence

The recovered `root : n100906` credentials authenticated successfully over SSH.

Impact

Reuse of application credentials for system-level access dramatically increases impact, converting a web vulnerability into a stable, interactive foothold on the operating system.

Remediation

- Ensure application accounts and OS/SSH accounts are separate and never share passwords.
- Prefer key-based SSH authentication and disable password authentication where possible.
- Restrict SSH exposure via firewalling and, where appropriate, multi-factor authentication.

Verbose Error / Stack-Trace Information Disclosure

F-05 · MEDIUM

Severity: MEDIUM | **Component:** Node.js/Express API, port 8081 | **Type:** Information Exposure Through Error Message (CWE-209)

Description

When the `/ping` endpoint was called without its expected parameter, the API returned a full, unhandled stack trace to the client. The trace disclosed the absolute source-file path, the exact line of vulnerable code, the framework and library versions in use, and the fact that user input was being processed via `.replace()` — information that directly guided exploitation of F-01.

Evidence

```
TypeError: Cannot read property 'replace' of undefined
    at app.get (/home/www/api/index.js:45:29)
    at Layer.handle [as handle_request] (.../express/lib/router/layer.js:95:5)
    at cors (/home/www/api/node_modules/cors/lib/index.js:188:7)
```

Impact

Leaked internal implementation details (paths, code structure, dependencies) significantly lower the effort required to discover and exploit other vulnerabilities.

Remediation

- Disable verbose/stack-trace error responses in production; return generic error messages to clients.
- Implement centralised error handling that logs details server-side only.

Information Disclosure via robots.txt / Sitemap

F-06 · LOW

Severity: **LOW** | Component: Web application, port 31331 | Type: Information Disclosure (CWE-200)

Description

The web application's `robots.txt` referenced a sitemap (`utech_sitemap.txt`) that enumerated application paths, including a `/partners.html` login page. While not directly exploitable, this expanded the discoverable attack surface.

Evidence

```
robots.txt → utech_sitemap.txt
/
/index.html
/what.html
/partners.html
```

Impact

Reduced effort for an attacker to map the application and locate sensitive pages.

Remediation

- Avoid listing sensitive or administrative paths in publicly accessible files.
- Protect sensitive endpoints with authentication and access control rather than obscurity.

6. Privilege Escalation

Following interactive SSH access, local privilege escalation was performed to obtain **root**, completing the host compromise.

Note on documentation completeness: the specific privilege-escalation vector was not captured in the source assessment notes. For a complete and reproducible report, this section should be expanded with the exact escalation technique used (e.g. the enumeration step that revealed it, the command executed, and proof of **root**). This is a useful reminder to record every step at the time of testing.

7. Remediation Summary

ID	PRIORITY ACTION	PRIORITY
F-01	Eliminate shell invocation in the API; validate the <code>ip</code> parameter against a strict allow-list.	IMMEDIATE
F-02	Move the database outside the application root and restrict file permissions.	HIGH
F-03	Re-hash all passwords with bcrypt/scrypt/Argon2 and force a reset.	HIGH
F-04	Separate application and OS credentials; prefer SSH keys, disable password auth.	HIGH

ID	PRIORITY ACTION	PRIORITY
F-05	Disable verbose error output in production; log details server-side only.	MEDIUM
F-06	Remove sensitive paths from public sitemap/robots files; enforce access control.	LOW

8. Appendix A — Services & Testing Performed

Discovered Services

PORT	SERVICE	VERSION / NOTES
21/tcp	FTP	vsftpd 3.0.5
22/tcp	SSH	OpenSSH 8.2p1 Ubuntu 4ubuntu0.13
8081/tcp	HTTP	Node.js (Express) — REST API (entry point)
31331/tcp	HTTP	Web application

Testing Performed (including ruled-out paths)

The following avenues were investigated and confirmed as dead ends. They are recorded for completeness, as ruling out a path is itself part of mapping the attack surface:

AVENUE TESTED	RESULT
FTP — anonymous login	Not permitted
FTP — credential brute force (hydra)	Unsuccessful
FTP — public exploit (searchsploit, vsftpd 3.0.5)	None applicable
SSH — credential brute force	Unsuccessful
Web (31331) — SQLi login bypass on /partners.html	Unsuccessful
API — user enumeration on /auth	No useful difference

Tester's note. This assessment was conducted manually without relying on automated AI guidance for attack-surface identification — the analysis of the stack trace, the mapping of the API surface, and the exploitation decisions were reasoned through directly. Documenting findings in this structured format is part of building a repeatable, independent testing methodology.